Übungsblatt 6: Hardware-Beschleunigung

Mikrocomputertechnik 3 · Datenbedarf, Pufferbesitz und Fristen auf Papier

Musterlösung

Ziel und Arbeitsweise

Die sechs Pflichtaufgaben sind ohne Platine oder Simulator auf Papier lösbar. Sie berechnen Grenzen, verfolgen Speicheradressen und begründen sichere Pufferübergaben. DMA bedeutet Direct Memory Access, direkter Speicherzugriff; CPU Central Processing Unit, Prozessor.

Bearbeitungszeit: etwa 60–75 Minuten selbstständig. **Orientierungspunkte:** 32. Im Unterricht sind 55 Minuten angeleitete Arbeitsphasen vorgesehen: Aufgabe 1 (10 min), Aufgaben 2–3 (15 min), Aufgabe 4 (15 min), Aufgaben 5–6 (15 min). Lösungsbesprechungen gehören zu den Vorlesungsphasen.

Geben Sie Rechenweg, Einheiten und Modellgrenzen an. Ein Byte hat acht Bits, MB bedeutet 10^6 Bytes, MiB 2^{20} Bytes. Hexadezimale Adressen beginnen mit `0x`. Geräteadressen, Zählergrenzen, Cachegrößen und Zeitwerte sind Lehrannahmen, keine zugesicherte Plattform.

Aufgabe 1: Leitung und CPU-Budget (6 Punkte)

Ein vollständiges Bild besitzt 320×240 Pixel im Format RGB565: Red, Green, Blue mit fünf, sechs und fünf Bits, insgesamt zwei Bytes je Pixel. Ein SPI, Serial Peripheral Interface, überträgt ideal ein Bit je SCK, Serial Clock oder Takt, ohne Pausen und Kommandobytes.

1. Berechnen Sie die Bildgröße und die Nutzdatenrate für 60 vollständige Bilder pro Sekunde. (2 P)
2. Berechnen Sie die idealen maximalen Bildraten bei 10 MHz und 50 MHz SCK. Kann DMA allein 60 Bilder/s ermöglichen? (1 P)
3. Die CPU hat 100 MHz. Bei 10 MHz SCK fallen modellhaft 150 CPU-Takte fester Interruptaufwand je Meldung an. Berechnen Sie den Bedarf bei einer Meldung je Byte und je sechzehn Bytes. Wie ist ein Bedarf über 100 % zu interpretieren? (2 P)
4. Bestimmen Sie die ideale Leitungszeit für 1 MiB bei 10 MHz SCK. Weshalb ergibt die Rechnung für 1 MB eine andere Zeit? (1 P)

Musterlösung 1

1. $B = 320 \cdot 240 \cdot 2 = 153600$ Bytes. $r = 60 \cdot 153600 = 9216000$ Byte/s, also 9,216 MB/s. Die Leitung müsste mindestens 73,728 Mbit/s Nutzdaten tragen. (je 1 P)
2. $10^7/8 = 1250000$ Byte/s, damit $1250000/153600 \approx 8{,}14$ Bilder/s. Bei 50 MHz sind es $\approx 40{,}69$ Bilder/s. Beide Werte liegen unter 60. DMA kann CPU-Kopierarbeit sparen, erhöht diese Leitungsgrenze aber nicht. Reale Zusatzbytes senken die Werte. (1 P)
3. Pro Byte: $1250000 \cdot 150 = 187500000$ Takte/s, also 187,5 % des verfügbaren Budgets. Das ist ein nicht erfüllbarer Bedarf, keine erreichbare CPU-Auslastung. Pro sechzehn Bytes: $1250000/16 = 78125$ Meldungen/s, damit 11718750 Takte/s oder 11,71875 %. Datenkopieren und Anwendungsarbeit sind darin noch nicht enthalten. (1 P Zahlen, 1 P Deutung)
4. $t = 2^{20} \cdot 8/10^7 = 0{,}8388608$ s. 1 MB enthält nur 1000000 Bytes und benötigt 0,8 s. Binäres und dezimales Präfix bezeichnen unterschiedliche Mengen. (1 P)

Aufgabe 2: Adressen, Elemente und Blöcke (6 Punkte)

Es gelten bereits DMA-sichtbare 32-Bit-Modelladressen und gleiche Quell- und Zielbreite. COUNT ist die Anzahl der **Elemente**, nicht generell die Bytezahl. Inkrement bedeutet Erhöhen um die Elementbreite.

1. Beim Senden von vier Bytes beginnt die Quelle bei `0x20001000`, das UART-TX-Register liegt bei `0x40010004`. Schreiben Sie die vier Paare aus Quell- und Zieladresse auf. UART heißt Universal Asynchronous Receiver-Transmitter, TX Transmit. (1 P)

2. Beim Empfang liegt ADC_DATA fest bei `0x40011000`. Acht 16-Bit-Messwerte sollen ab `0x20002000` gespeichert werden. Nennen Sie die ersten vier Adresspaare, die Gesamtbytezahl und die Zieladresse des letzten Elements. ADC heißt Analog-to-Digital Converter, Analog-Digital-Wandler. (2 P)

3. Ein Byte-Transfer ist auf 65535 Elemente je Auftrag begrenzt. Zerlegen Sie 1000000 Bytes in möglichst wenige Aufträge. Wie viele volle Blöcke und welche Restlänge entstehen? (1 P)

4. Ermitteln Sie die Rückgabe der folgenden C-Funktion für `(70000, 65535)`, `(20, 65535)` und `(0, 65535)`. Welche Gefahr bleibt bei `limit = 0` für eine Schleife, die erst nach einem positiven Block fortschreitet? (1 P)

```c
uint16_t next_block(uint32_t remaining, uint16_t limit) {
    if (remaining < limit)
        return (uint16_t)remaining;
    return limit;
}
```

5. Ein 32-Bit-Zugriff verlangt hier eine durch vier teilbare Geräteadresse. Prüfen Sie `0x20001002` und `0x20001004`. Erzeugt ein C-Cast eines CPU-Zeigers automatisch eine gültige DMA-Geräteadresse? Begründen Sie kurz. (1 P)

`remaining` ist die verbleibende Elementzahl, `limit` die positive Auftragsgrenze. `uint16_t` und `uint32_t` sind vorzeichenlose 16- beziehungsweise 32-Bit-Typen aus `<stdint.h>`. Eine Rückgabe null erlaubt **keinen** Hardwarestart; dessen Zählersemantik müsste gesondert dokumentiert sein.

Musterlösung 2

1. Quellen `0x20001000`, `0x20001001`, `0x20001002`, `0x20001003`; Ziel jeweils `0x40010004`. Die Quelle wächst um ein Byte, das einzelne Senderegister bleibt fest. (1 P)
2. Quelle jeweils `0x40011000`; erste Ziele `0x20002000`, `0x20002002`, `0x20002004`, `0x20002006`. Acht Elemente zu zwei Bytes sind sechzehn Bytes. Letztes Element beginnt bei `0x2000200E` und belegt auch `...200F`. (je 1 P)
3. $15 \cdot 65535 = 983025$ Bytes in fünfzehn vollen Blöcken; Rest 16975 Bytes. Insgesamt sechzehn Aufträge. (1 P)
4. Rückgaben 65535, 20 und 0. Ein Limit null liefert auch bei Restdaten null; die verbleibende Menge nimmt dann nicht ab. Der Aufrufer muss ein positives Limit sicherstellen und bei Rest null aufhören. Der Cast im kleineren Zweig ist sicher, weil dort `remaining < limit <= 65535` gilt. (1 P)
5. `...1002` ist nicht durch vier teilbar, `...1004` ist es. Ein Cast verändert nur die C-Darstellung. Er bildet keinen Speicher für das Gerät ab und prüft weder Erreichbarkeit noch Plattformübersetzung. (1 P)

Aufgabe 3: Wann gehört der Puffer wieder der CPU? (5 Punkte)

Für einen endlichen normalen Sendeauftrag dokumentiert der Treiber: DMA-TC, Transfer Complete, bestätigt die erfolgreiche letzte Speicherbewegung und das Ende weiterer Zugriffe auf den Quellpuffer. Ein Peripherie-FIFO, First In, First Out, und ein Schieberegister können anschließend noch Daten an die Leitung abgeben.

1. Die CPU verändert den Quellpuffer, während DMA daraus liest. Warum ist das trotz gültiger Speicheradresse ein Fehler? (1 P)
2. Ein SPI-Gerät verlangt aktives CS, Chip Select, bis das letzte Taktbit übertragen wurde. Darf CS unmittelbar bei DMA-TC deaktiviert werden? Welche zweite Bedingung ist nötig? (1 P)
3. Ist der Quellpuffer unter der obigen normalen Abschlussgarantie bei DMA-TC wieder nutzbar? Wie unterscheidet sich das von TC im Circular Mode, dem fortlaufenden Umlaufbetrieb? (1 P)
4. Ein Timeout läuft ab, bevor der Auftrag erfolgreich abgeschlossen ist. Formulieren Sie den Abbruchweg einschließlich einer zweiten, begrenzten Wartefrist. Was passiert mit dem Puffer, wenn ruhender DMA nicht bestätigt werden kann? (2 P)

Musterlösung 3

1. DMA kann eine Mischung aus altem und verändertem Inhalt lesen. Die Adresse ist erreichbar, aber der vereinbarte Pufferbesitz verletzt. Die CPU hält die veröffentlichte Quelle bis zum Ende aller Zugriffe unverändert. (1 P)
2. Nein. Erst das dokumentierte Leitungsende der SPI-Peripherie, einschließlich des letzten Taktbits, erlaubt die geforderte CS-Freigabe. DMA-TC bestätigt nur die letzte Speicherbewegung. (1 P)
3. Unter der ausdrücklich gegebenen normalen Erfolgsgarantie ja: Die nachlaufende Peripherie hält bereits eigene Daten. Im Circular Mode kann TC dagegen nur einen Umlauf markieren; DMA setzt wieder am Anfang an. Der ganze Puffer ist dann nicht frei. (1 P)
4. Neue Aufträge verhindern, Stopp über den Plattformweg anfordern, innerhalb einer begrenzten Frist bestätigen, dass keine Zugriffe mehr erfolgen, anschließend Status und Peripherie geordnet behandeln. Erst bestätigter Stillstand erlaubt die Puffer-Rückgabe. Ein unbestätigt weiter aktiver Kanal behält einen gesperrten Puffer; bloßes Verlassen der Wartefunktion erlaubt keine Wiederverwendung. Unbestätigte Empfangsteile gelten nicht als neuer vollständiger Datensatz. (je 1 P)

Aufgabe 4: Sichtbarkeit und ganze Cachezeilen (6 Punkte)

Es gilt ein **nicht kohärenter Write-Back-Daten-Cache**: Neue CPU-Werte können zunächst nur in einer lokalen Kopie liegen; DMA greift auf den Speicher dahinter zu. Die Cachezeile hat 32 Bytes. Clean schreibt Dirty-Daten zurück; Invalidate verwirft die Cachekopie. Dirty heißt geändert und noch nicht zurückgeschrieben.

1. Beim Senden enthält der Speicher das Hexbyte `11`, die CPU hat im Cache `22` geschrieben, die Zeile ist Dirty. Was liest DMA ohne Wartung? Beschreiben Sie eine sichere Übergabe. (1 P)
2. Beim Empfang bleibt eine alte Dirty-Kopie `AA` im Cache, DMA schreibt `55` in den Speicher. Was könnte ein CPU-Cache-Hit liefern, und was bewirkt späteres Write-Back? Warum genügt blindes Clean nach dem Empfang nicht? (2 P)
3. Ein Puffer beginnt bei `0x20001003` und umfasst 100 Bytes. Bestimmen Sie das exklusive Ende, die letzte enthaltene Byteadresse, den vollständigen Cache-Wartungsbereich und dessen Länge. Wie viele Bytes außerhalb des Puffers werden dabei mitbetroffen? (2 P)
4. Die Randbytes enthalten fremde Dirty-Variablen. Nennen Sie eine sichere Layoutregel. Ersetzt ein RISC-V-`fence` automatisch die Cache-Wartung? Reicht ein Linkerbereich namens `RAM_UNCACHED` als Beweis für einen geeigneten DMA-Puffer? (1 P)

Ein exklusives Ende bezeichnet die erste **nicht** enthaltene Adresse. RISC-V ist die Prozessor-Befehlssatzarchitektur. `fence` ordnet Speicherzugriffe gemäß seiner Plattformverwendung; `RAM_UNCACHED` ist zunächst nur ein gewählter Bereichsname.

Musterlösung 4

1. DMA liest `11`. Vor Übergabe die neuen `22` gemäß Plattformregel in die DMA-sichtbare Ebene veröffentlichen, nötige Reihenfolge sichern und die Quelle während DMA-Zugriffen unverändert halten. (1 P)
2. Ein Cache-Hit liefert noch `AA`; späteres Write-Back kann die neu empfangenen `55` mit `AA` überschreiben. Ein spätes Clean würde genau diese alten Daten veröffentlichen. Empfangsbereiche werden vor Übergabe nach Plattformregel vorbereitet und nach bestätigtem Empfang zur CPU-Sicht synchronisiert. Das konkrete API, Application Programming Interface, muss festlegen, welche Wartung wann nötig und erlaubt ist. (1 P Fehlerfolge, 1 P Übergabe)
3. Exklusives Ende `0x20001067`, letzte Byteadresse `0x20001066`. Nach unten beziehungsweise oben auf Zeilengrenzen: `0x20001000` bis ausschließlich `0x20001080`. Das sind 128 Bytes, vier Cachezeilen. Drei Bytes davor und fünfundzwanzig danach sind mitbetroffen. (je 1 P)
4. Ganze ausgerichtete Cachezeilen exklusiv reservieren, einschließlich des aufgerundeten Endes; keine fremden Dirty-Daten in diesen Zeilen. `fence` schreibt nicht automatisch einen Write-Back-Cache zurück. Der Linkername konfiguriert keine Cachebarkeit. Plattformattribute, DMA-Erreichbarkeit und eindeutiger Pufferbesitz sind separat zu prüfen. (1 P)

Aufgabe 5: Ping-Pong-Frist und Ringrückstand (5 Punkte)

Ein Mono-Audiostrom enthält 44100 Samples pro Sekunde mit 16 Bit je Sample. Ein Ping-Pong-Sendepuffer besitzt zwei Hälften zu je 2048 Bytes. DMA liest erst A, dann B, dann wieder A. Die CPU darf die jeweils verlassene Hälfte nachfüllen.

1. Berechnen Sie Datenrate und Halbzeit T_H. Ein Nachfüllen benötigt höchstens 3 ms Reaktionslatenz, 18 ms Arbeit und 1 ms Veröffentlichung. Passt das Fenster? Reichen 30 ms Arbeit allein? (2 P)
2. HT, Half Transfer, wird beim ersten Wechsel A nach B gesetzt. Die CPU bearbeitet dieses Flag erst bei $2T_H + 1$ ms. Welche Hälfte liest DMA dann? Erlaubt das alte HT-Flag einen Schreibzugriff auf A? (1 P)
3. Ein unabhängiger Empfangsring hat $N = 8$ Plätze. Konsistent bekannte Gesamtzähler melden $P = 14$ geschriebene und $C = 5$ verarbeitete Samples; die Samples sind ab null nummeriert. Berechnen Sie Rückstand, beide Ringpositionen und deren Differenz modulo acht. Welches unverarbeitete Sample ist bereits überschrieben? (2 P)

`P` und `C` sind hier **gegebene Gesamtzähler**, keine automatisch aus einem einzelnen Flag ableitbaren Werte. Ein gesetztes Statusbit kann mehrere Ereignisse zusammenfassen.

Musterlösung 5

1. $r = 44100 \cdot 2 = 88200$ Byte/s. $T_H = 2048/88200 \approx 23{,}21995$ ms. $3 + 18 + 1 = 22$ ms passen unter den angenommenen Höchstwerten: Reserve $\approx 1{,}21995$ ms. Dreißig Millisekunden Arbeit allein überschreiten das Fenster. Die Frist muss für jeden Umlauf gelten, nicht nur im Mittel. (je 1 P)
2. Nach $2T_H \approx 46{,}43991$ ms liest DMA erneut A. Bei $\approx 47{,}43991$ ms ist A deshalb belegt. Das verspätete Flag berichtet einen früheren Übergang, keine aktuelle Schreibfreigabe. (1 P)
3. $P - C = 9 > 8$: Der Ring reicht nicht aus. Schreibposition $14 \bmod 8 = 6$, Leseposition $5 \bmod 8 = 5$; Positionsdifferenz $(6-5) \bmod 8 = 1$ verschleiert den Rückstand neun. Sample 13 hat im Platz fünf das noch unverarbeitete Sample 5 überschrieben. Plätze null bis sieben enthalten jetzt $8, 9, 10, 11, 12, 13, 6, 7$. Gesamtfortschritt ist nötig, wenn Überläufe sicher erkannt werden sollen. (je 1 P)

Aufgabe 6: Rechtecktransfer und begründete Auswahl (4 Punkte)

Ein RGB565-Bild mit zwei Bytes je Pixel hat 320 Pixel je Zeile und beginnt bei `0x20001000`. Die Koordinaten starten bei null. Ein Rechteck ab $x = 10$, $y = 20$ mit vier Pixeln Breite und drei Zeilen Höhe soll ab `0x20008000` dicht gepackt werden. Pitch ist der Byteabstand zwischen zwei Zeilenanfängen.

1. Bestimmen Sie Quell-Pitch, kopierte Bytes je Zeile, Abstand vom Ende einer kopierten Zeile zum nächsten Quellanfang und Gesamtbytezahl des Rechtecks. (1 P)
2. Berechnen Sie die drei Quell- und Zielanfangsadressen. Warum genügt ein einziger linearer 24-Byte-Auftrag aus der Quelle nicht? Nennen Sie eine passende Zerlegung. (2 P)
3. Ein anderer Sensor liefert nur sechs Bytes je 10 ms. Muss dafür DMA gewählt werden? Begründen Sie die Auswahl mit CPU-Aufwand, Frist und Komplexität. (1 P)

Musterlösung 6

1. Quell-Pitch $320 \cdot 2 = 640$ Bytes, kopierte Zeile $4 \cdot 2 = 8$ Bytes, Lücke nach diesen acht Bytes $640 - 8 = 632$ Bytes. Drei kopierte Zeilen ergeben 24 Bytes. Pitch und Lücke sind verschieden. (1 P)
2. Startoffset $20 \cdot 640 + 10 \cdot 2 = 12820 =$ `0x3214`. Quellen `0x20004214`, `0x20004494`, `0x20004714`. Ziele `0x20008000`, `0x20008008`, `0x20008010`. Die nächsten Quellzeilen liegen 640 Bytes auseinander; ein linearer Auftrag kopierte überwiegend Nachbarpixel derselben Zeile. Drei lineare Aufträge zu acht Bytes passen. Alternativ sind dokumentierte Deskriptorketten oder ein geeigneter zweidimensionaler DMA-Modus möglich. Deren Verfügbarkeit muss geprüft werden. (je 1 P)
3. Nein, 600 Byte/s allein verlangen keinen DMA. Ein kurzer blockierender Aufruf oder Interruptbedienung kann bei eingehaltenen Reaktionsfristen einfacher sein. DMA kann sich dennoch wegen anderer CPU-Arbeit oder Energiezielen lohnen. Entscheidend sind geprüfter Aufwand und Fristen, nicht nur die Existenz eines Kanals. (1 P)

Optional: zwei reine C-Rechenfunktionen am Host prüfen

Diese Vertiefung benötigt einen C-Compiler am normalen Rechner. Sie prüft **keinen** Hardwaretreiber und führt keine Gerätezugriffe aus. Speichern Sie die folgende Datei als `dma_rechnen.c`.

```
#include <stdint.h>
#include <inttypes.h>
#include <stdio.h>

static uint16_t next_block(uint32_t remaining, uint16_t limit) {
    if (remaining < limit)
        return (uint16_t)remaining;
    return limit;
}

static uint64_t covered_end(uint32_t start, uint32_t bytes) {
    uint64_t end = (uint64_t)start + bytes;
    return (end + 31u) & ~UINT64_C(31);
}

int main(void) {
    printf("Block: %" PRIu16 "\n", next_block(1000000u, 65535u));
    printf("Ende: 0x%08" PRIX64 "\n",
           covered_end(UINT32_C(0x20001003), 100u));
    return 0;
}
```

Bauen und starten Sie beispielsweise mit:

```
gcc -std=c11 -Wall -Wextra -Werror dma_rechnen.c -o dma_rechnen
./dma_rechnen
```

Erwartet werden `Block: 65535` und `Ende: 0x20001080`. Ändern Sie die Eingaben auf `remaining = 0`, `limit = 0` sowie `start = 0x20001000`, `bytes = 32`. Erklären Sie die Ergebnisse.

`covered_end` berechnet ausschließlich die Geometrie eines positiven Bytebereichs mit 32 Bytes je Cachezeile. `start` ist dessen Anfang, `bytes` seine Länge und `end` das exklusive Ende. `uint64_t` ist ein vorzeichenloser 64-Bit-Typ. Der Cast erfolgt **vor** der Addition; `UINT64_C(31)` erzeugt die Maske in derselben Breite. `~` invertiert Bits, `&` verknüpft sie bitweise. Die Druckmakros `PRIu16` und `PRIX64` aus `<inttypes.h>` passen das Ausgabeformat an die Typen an.

Der Aufrufer prüft vorher positive Länge und gültigen Gerätebereich. Null Bytes bedeuten keinen Wartungsauftrag. Die Rückgabe ist kein Beweis für Erreichbarkeit, exklusive Cachezeilen oder eine ausgeführte Cache-Wartung. Ein passender Hardwaretreiber benötigt zusätzlich dokumentierte Request-Zuordnung, Registersemantik, Adressabbildung, Synchronisation, Fehlerbehandlung und ein bestätigtes Ende aller Zugriffe.